

Aggregation by Group

Overview

- [Split-Apply-Combine](#)
- [Categorical data](#)
- [Pandas .groupby\(\) method](#)
- [Re-merging](#)
- [Aggregation Functions](#)
- [Generalized Split-Apply-Combine with .apply\(\)](#)
- [Takeaways](#)

Split, Apply, Combine

- A common task in data analysis is to compute some summary statistic (an aggregation) for disjoint subsets (subgroups) in the data.

Split, Apply, Combine

- This task is often referred to as *split-apply-combine* b/c we: split the data into groups, apply an aggregation function to each group, and then combine the results into a new data frame or column(s).

Example Data

- Here is a small example dataset we'll use in these slides.

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(9 * 2021 * 28)
n=100
a = rng.binomial(n=1, p=0.5, size=n)
b = 1 - 0.5 * a + rng.normal(size=n)
c = 0.8 * a + rng.normal(size=n)
df = pd.DataFrame({'a': a, 'b': b, 'c': c})
df
```

Categorical Data

- Grouping is most commonly done with categorical data.
- Categorical data are often coded as integers having associated labels.
- Panda's `pd.categorical()` can be used to create a categorical type.

```
df['a'] = pd.Categorical(df['a'].replace({0: 'control', 1: 'treatment'}))  
(df['a'].dtype, df['a'].values.categories, df['a'].values.codes)
```

Pandas .groupby() method

- The split-apply-combine or *aggregation by group* paradigm is implemented in pandas as the `.groupby()` method.
- We'll focus on grouping by variables in the data; you'll read about other ways of grouping.
- We'll also limit our focus to grouping rows, but columns can be grouped too.

```
df.groupby('a').size()
```

Pandas .groupby() method

- By default, the columns used to define group membership become the index in the resulting DataFrameGroupBy object.

```
[df.groupby('a').mean().index, df.groupby('a').mean().columns]
```

Pandas .groupby() method

- By default, the columns used to define group membership become the index in the resulting DataFrameGroupBy object.
- You can request the group keys as value columns using `as_index=False`.

```
df.groupby('a', as_index=False).mean().columns
```

Re-merging

- *Re-merging* is the name for a technique in which aggregation by group is used to compute summary statistics which are then (re-)merged with the source data.

Re-merging

- For example, you may want to compute a mean/min/max by group and then broadcast to a new column.

```
df_max_a = (  
    df  
    .groupby('a')[['b']]  
    .max()  
    .rename(columns={'b': 'b_max'})  
    )  
df2 = df.set_index('a').join(df_max_a)  
df2.groupby(level='a').head(n=2)
```

Indexing

- In the previous example (copied below), note that we indexed the DataFrameGroupBy object to limit the columns operated on.

```
df_max_a = (  
    df  
    .groupby('a')[['b']]  
    .max()  
    .rename(columns={'b': 'b_max'})  
    )  
df2 = df.set_index('a').join(df_max_a)  
df2.groupby(level='a').head(n=2)
```

Re-merging

- *Re-merging* is a very general technique.
- In pandas this can also be accomplished using `.groupby()` with `.transform()`.

```
df3 = df.copy()
df3[['b_max', 'c_max']] = (
    df3
    .groupby('a')
    .transform(np.max)
)
df3.groupby('a').head(n=2)
```

Aggregation functions

- Common aggregation functions have been optimized for groupby and are available as methods (see Table 10-1 in McKinney): sum, mean, std, var, median, min, max, first, last, count (compare to size).

```
df3.iloc[1, 1] = np.nan  
df3.groupby('a').size()  
#df3.groupby('a').count()
```

General Aggregation functions

- The `.agg()` (`.aggregate()`) method supports more general functions.

```
(  
df  
  .groupby('a')  
  .agg(lambda x: np.quantile(x, .75) - np.quantile(x, .25))  
  .rename(mapper=lambda x: 'iqr_' + x, axis=1)  
)
```

General Aggregation functions

- You can use a list or a list of tuples with `.agg()`.

```
f_list = [  
    ('min', np.min),  
    ('max', np.max),  
    ('iqr', lambda x: np.quantile(x, .75) - np.quantile(x, .25)),  
]  
df.groupby('a').agg(f_list)
```

Aggregation functions

- Series methods can also be used with *GroupBy objects.

```
df.groupby('a').quantile([.025, .975])
```

General Split-Applied-Combine

- For more general tasks, the `.apply()` method operates on each subset of data and then puts them back together.
- Useful especially when implementing multi-column logic.
- In this example, we keep all rows where b or c is in the top or bottom 2.5% within each group.

General Split-ApPLY-Combine

- Here is a function that implements the desired subset behavior for a DataFrame.

```
def tail_values(df, columns=None, lwr=.025, upr=.975):
    """
    Subset a DataFrame df to find rows with values in the distributional tail.

    Parameters
    -----
    df : DataFrame
        The DataFrame to be subset.
    columns : string or list of strings. Optional.
        Names of columns in which to look for tail values. If None use all
        columns. The default is None.
    lwr, upr : float. Optional.
        Sample quantiles (inclusive) demarking the lower and upper tails,
        respectively. The defaults are .025 and .975.

    Returns
    -----
    A subset of df with rows taking values in the distributional tail of any
    column in columns.
    """
    if columns is None:
        tail = df.transform(lambda x: (
            np.logical_or(x >= np.quantile(x, upr), x <= np.quantile(x, lwr))
        )).any(axis=1)
    elif isinstance(columns, list):
        tail = df[columns].transform(lambda x: (
            np.logical_or(x >= np.quantile(x, upr), x <= np.quantile(x, lwr))
        )).any(axis=1)
    elif isinstance(columns, str):
        tail = df[[columns]].transform(lambda x: (
            np.logical_or(x >= np.quantile(x, upr), x <= np.quantile(x, lwr))
        )).any(axis=1)
    else:
        raise TypeError("columns should be a str, list or None.")

    return(df[tail])
```

General Split-Apply-Combine

- And now we apply the function to each group.

```
df.groupby('a').apply(tail_values)
```

Takeaways 1/2

- *Split-Apply-Combine* or *aggregation by group* is implemented in the `.groupby()` method of pandas DataFrame (Series) class.
- Grouping variables become an index by default; control with `as_index`.
- Create new columns with group-wise summary statistics using `.groupby()` and `.transform()` or by re-merging.

Takeaways 2/2

- Use optimized aggregation functions when available and the `.agg()` method when not.
- For more complex operations, the `.apply()` method operates on each group and then puts the pieces back together.
- Use this and other DataFrame methods whenever you can.